

Лабораторная работа №5

Аппарат сетей Петри

Разанацуа Сара Естэлл

Российский университет дружбы народов, Москва, Россия

.. ..
.. ..

- Построим сеть Петри для пяти философов, моделируя захват и освобождение вилок. Обнаружим состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую. Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock. Модифицировать сеть, чтобы предотвратить deadlock. Проанализировать результаты и оформить отчёт с графиками и анимацией.

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
 - чистый код;

Выполнение лабораторной работы

- Мы создадим файл `src/DiningPhilosophers.jl` и запустим его.

```
Ouvrir ▾  DiningPhilosophers.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/labs/lab05/project/src
dining_philosophers.jl | dining_philosophers_animation.jl

module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra
|

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],

```

- Вывод в консоль.

```
julia> touch("src/DiningPhilosophers.jl")
"src/DiningPhilosophers.jl"

julia> include("src/DiningPhilosophers.jl")
Main.DiningPhilosophers
```

Рис. 2: Рис

- Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:
 1. классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
 2. модифицированная модель с арбитром, которая должна предотвращать deadlock.



```
using DrWatson
@quickactivate "project"
include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots

N = 5
tmax = 10.0

println("==== Классическая сеть (без арбитра) ====")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)

println("Deadlock observed: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotdir("classic_simulation.png"))
println("==== Сеть с арбитром ====")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock observed: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotdir("arbiter_simulation.png"))
```

Рис. 3: Рис

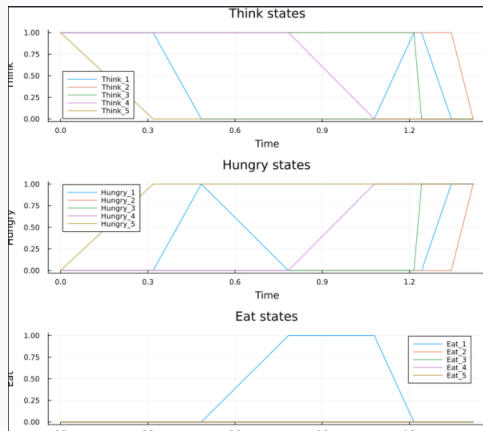
- Создание файла `scripts/dining_philosophers.jl` и запустим его.

```
julia> include("scripts/dining_philosophers.jl")
Activating project at `~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project`
WARNING: replacing module DiningPhilosophers.
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

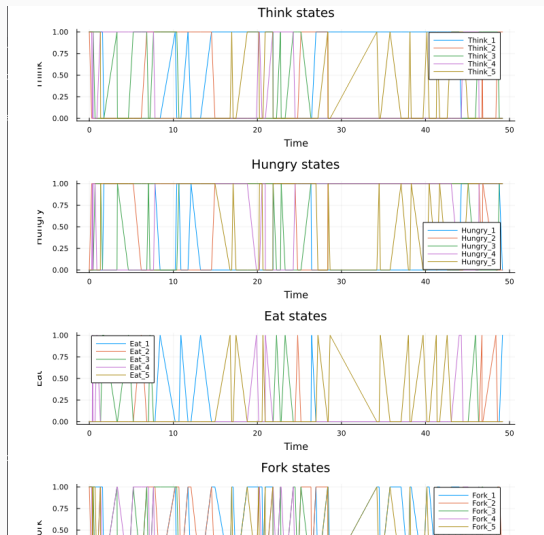
=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/serazanacua/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

Рис. 4: Рис

- Графики эволюции маркировки позволяют визуальнo увидеть, как меняются состояния.
- В классической сети рано или поздно все Eat_i становятся нулевыми, а $Hungry_i$ – единичными (или наоборот).



2. В сети с арбитром наблюдается постоянное чередование приёма пищи.



- Для наглядной демонстрации динамики работы сети Петри во времени.
- Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели.



```
using DrWatson
@quickactivate "projects"
include(sourced("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random

N = 3
tmax = 38.8
net, u0, names = build_classical_network(N)

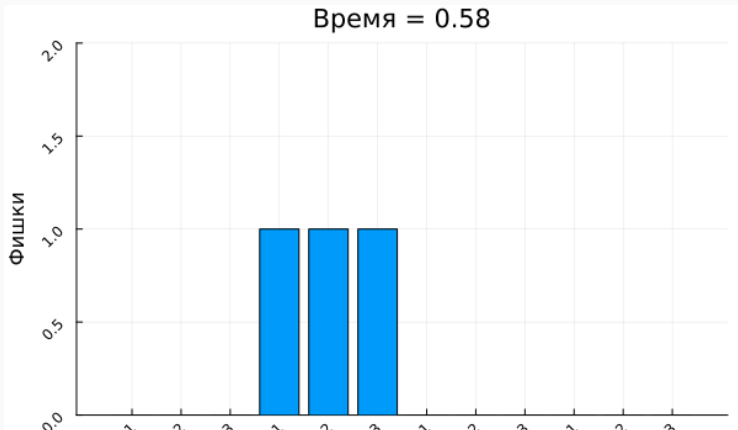
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u),
        u, legend = false,
        ylims = (0, maximum(u) + 1),
        xlabel = "Позиция",
        ylabel = "Физики",
        title = "Состояние системы в момент времени $(round(row.time, digits=2))",
    )
    xticks(1:length(u), string.(names), rotation = 45)
end

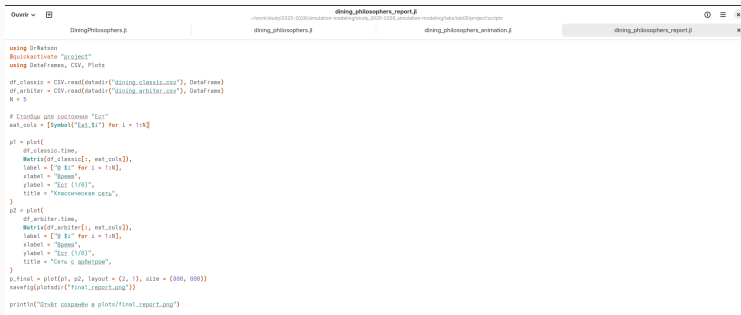
gif(anim, plotdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
```

Рис. 7: Рис

- Анимация делает абстрактные состояния сети Петри живыми.
- Видно, как фишки перемещаются между позициями: философы переходят из Think в Hungry, затем в Eat и обратно.



- Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i).
- Скрипт генерирует сводный график.



```
dining_philosophers_report.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026/simulation-modeling/tasks/lab05/project/scripts

DiningPhilosophers.jl | dining_philosophers.jl | dining_philosophers_animation.jl | dining_philosophers_report.jl x

using DrWatson
@activate "project"
using DataFrames, CSV, Plots

df_classic = CSV.read(pwd("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(pwd("dining_arbiter.csv"), DataFrame)
N = 5

# Создаём две матрицы "Ест"
eat_cols = [Symbol("Eat_$i") for i = 1:N]

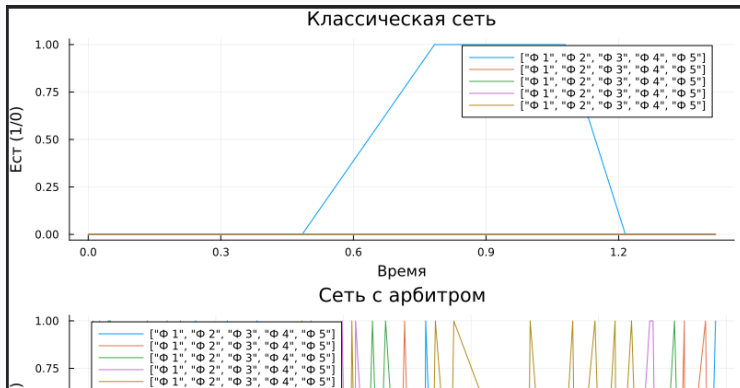
p1 = plot(
    df_classic.time,
    Matrix{Float64}(df_classic[:, eat_cols]),
    label = ["p $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая версия",
)
p2 = plot(
    df_arbiter.time,
    Matrix{Float64}(df_arbiter[:, eat_cols]),
    label = ["p $i" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Версия с арбитром",
)
p, final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotdir("final_report.png"))

println("Готово! Сохранено в plots/final_report.png")
```

Рис. 9: Рис

1. Классическая сеть: на графике видно, что через некоторое время все линии `Eat_i` падают до нуля и остаются на нуле
 - Это и есть deadlock.
 - Философы больше не могут есть.

2. Сеть с арбитром: линии Eat_i колеблются, всегда есть хотя бы один философ, который ест (или они поочерёдно получают доступ).
- Отсутствие длительных нулевых периодов подтверждает, что система жива и не блокируется.



- Я выполнила лабораторную работу.